



HACKTHEBOX



Intergalactic Recovery

30th November 2022 / Document No. D22.102.215

Prepared By: c4n0pus

Challenge Author(s): Bquanman

Difficulty: **Easy**

Classification: Official

Synopsis

Intergalactic Recovery is an easy forensic challenge based on RAID 5 data recovery. Featured in Cyber Apocalypse of 2022. Players must recover the missing data from a corrupt disk image based on parity data from the other two disk images.

Description

Miyuki's team stores all the evidence from important cases in a shared RAID 5 disk. Especially now that the case IMW-1337 is almost completed, evidences and clues are needed more than ever. Unfortunately for the team, an electromagnetic pulse caused by Draeger's EMP cannon has partially destroyed the disk. Can you help her and the rest of team recover the content of the failed disk?

Skills Required

- General Computer Science Understanding

Skills Learned

- Basic RAID implementations

Enumeration

We are given three disk images (`.img`) and also the information that these disk images are part of a RAID 5 group.

To understand this challenge, we will have to learn the basics of RAID before taking a crack at it:

RAID (Redundant Array of Independent Disks) is a configuration that combines multiple physical disks into a logical one, providing redundancy and more storage depending on the configuration.

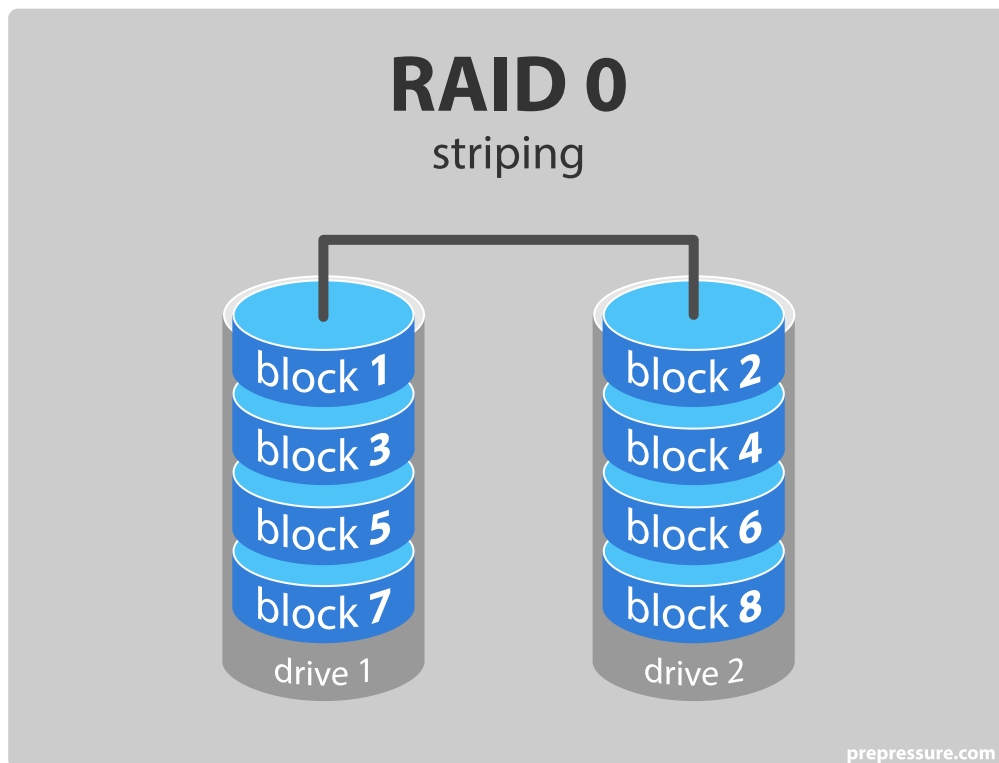
There are three main types of RAID nowadays:

- **RAID 0**
- **RAID 1**
- **RAID 5**

RAID 0

RAID 0 requires two or more drives (even of various sizes) striped together, meaning the total available size is the sum of the drives' capacity (capped at the smaller one). This arrangement features no parity and no redundancy. If one drive fails **ALL** of the data is gone.

IE: Let's say we have three drives with capacities of 20, 20, and 30 GBs, respectively. A RAID 0 arrangement of those drives would yield a total partition size of 60GBs with 10 GB left over from the 3rd drive.

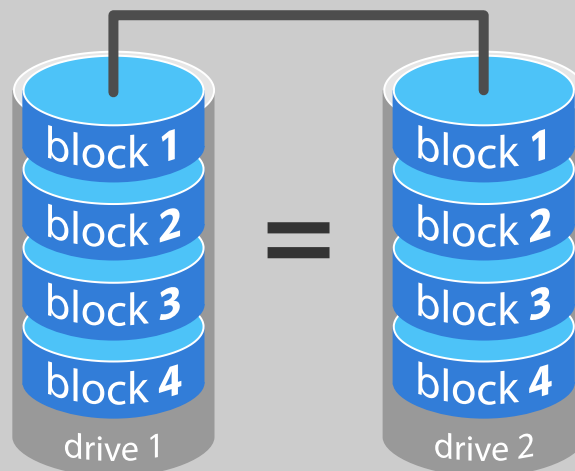


RAID 1

Opposite of RAID 0 is RAID 1, which mirrors the drives, with no parity nor striping. All blocks are mirrored to the other disk, and in the event that one of the disks fails, the data would still be safe and will allow time for replacing the failed drive. Expectedly, the total available space is reduced in half because of the mirroring.

RAID 1

mirroring



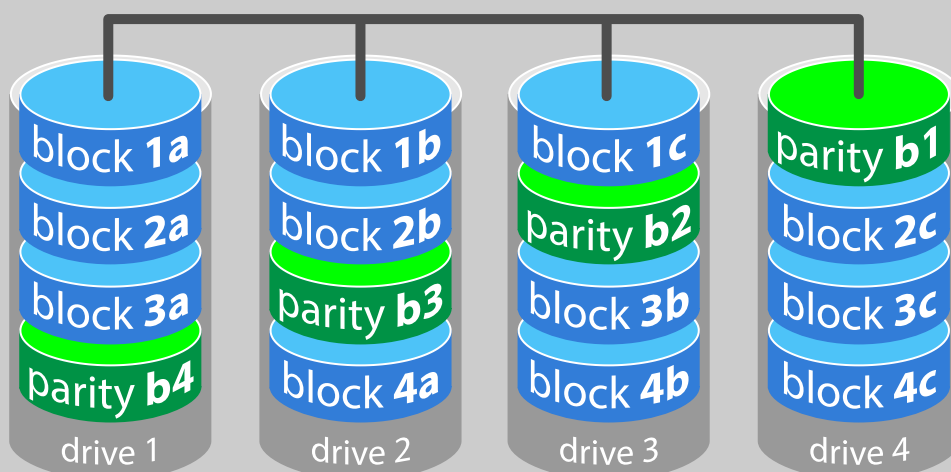
prepressure.com

RAID 5

And finally, we have RAID 5, which requires at least three disks. The basic principle of RAID 5 is storing data along with its parity bits. Eventually, we lose the capacity of 1 drive that is used for storing the parity bits. This all happens across all drives (in essence, not one drive is used to store all parity data, but the parity is split across all drives just like the data)

RAID 5

striping with parity across drives



prepressure.com

The idea is that we can lose up to 1 drive, and if so, on subsequent read operations, missing data can be produced using the parity bits on the fly. If we lose more than one drive, then **ALL** of our data is gone.

RAID arrays are more complex than that, and more elaborate configurations exist, but the above primer covers the three essential concepts (striping, mirroring, and parity).

Challenge

We know that our three disk images are part of a RAID 5 array, and by looking at their size, we can see that two of them are 5 MBs in size, and the last one is just under 4KBs. We suspect at this point that the 3rd disk is corrupted.

However, being part of a RAID 5 means we can calculate the data in that disk image.

RAID 5 uses the binary operation `XOR`, to calculate parity (and subsequently to calculate the missing data).

Solution

To recover the 3rd disk image, we must `XOR` the two healthy drives together to produce the missing disk image.

Then we need only to combine them into a logical drive to view the data.

The script below xors every byte from the other two disk images and produces the missing one:



```
1 d1 = open("./fef0d1cd.img", "rb").read()
2 d2 = open("./06f98d35.img", "rb").read()
3
4 data = b"".join([bytes([x^y]) for x,y in zip(d1,d2)])
5
6 with open("./recovered.img", "wb") as f:
7     f.write(data)
```

Now we have to reconstruct the RAID 5 configuration with our drives.

Firstly we are going to use `losetup` to make working with disk images easier:



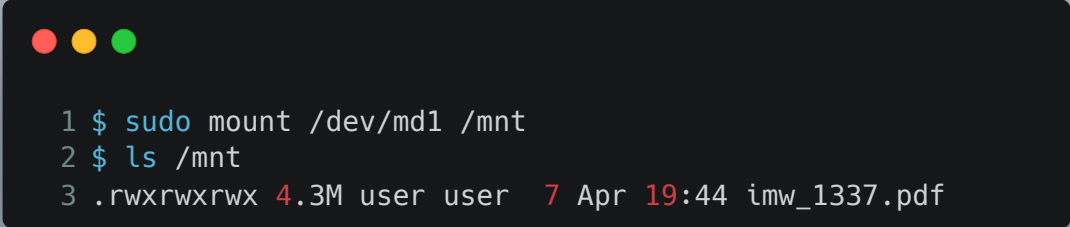
```
1 $ sudo losetup /dev/loop1 healthy_1.img
2 $ sudo losetup /dev/loop2 healthy_2.img
3 $ sudo losetup /dev/loop3 recovered.img
```

After which, we can use the program [mdadm](#) to set up our RAID 5 array.



```
1 $ sudo mdadm --create /dev/md1 --level=5 --raid-devices=3 /dev/loop1 /dev/loop2 /dev/loop3
```

Now we can mount our new device `/dev/md1` as a normal device and view the contents:



```
1 $ sudo mount /dev/md1 /mnt
2 $ ls /mnt
3 .rwxrwxrwx 4.3M user user 7 Apr 19:44 imw_1337.pdf
```

If we read the PDF we get the flag!